

FOURIER SAMPLING IN SIX LINES

DAVID WAKEHAM

ABSTRACT. We implement Fourier sampling—the quantum primitive underlying Shor’s algorithm—in six lines of **yaw** code, and map them directly onto the math. We use a pedagogical simplification of the algorithm which dispenses with the auxiliary register, and in an appendix, generalize it to Fourier sampling for the abelian Hidden Subgroup Problem.

Algorithm	Math
<code>def fourier_sample(p,d):¹</code>	For $G = \mathbb{Z}_d$, choose $pG \leq G$ with $p d$. ⁷
<code> \$alg = qudit(d)²</code>	Define $X^d = Z^d = I, \{Z, X\}_\times = \omega_d$. ⁸
<code> U = norm([Z**j for j in range(0,d,d//p)])³</code>	Construct oracle $U = \frac{1}{\sqrt{p}} \sum_{j=0}^{p-1} Z^{jd/p}$. ⁹
<code> circ = qft(X, Z) » U⁴</code>	Fourier-transform oracle, $\mathcal{F}[U]$. ¹⁰
<code> _, sample = measure(Z, circ « char(Z, 0))()⁵</code>	Apply $\mathcal{F}[U]$ to π_0 and measure. ¹¹
<code> return sample⁶</code>	Output is a random multiple of d/p . ¹²

ACKNOWLEDGMENTS

Thanks to Nabokov for loan of a framing device, will return after use.

¹As the syntax suggests, **yaw** is a Python-based language for quantum computing. It uses an algebraic approach to quantum programming, and naturally blends Pythonic and functional elements. For more details, feel free to visit github.com/torsor-io/yaw, download the code, and play around. See Appendix A for a reference implementation of period-finding in Qiskit (41 lines).

²The global variable `$alg` specifies an algebraic context, while `qudit(d)` is syntactic sugar for the generalized Clifford algebra $\mathcal{A}_d$, analogous to the Pauli algebra for qudits. For $d \geq 2$ the implementation is currently numerical, but future versions will allow for either fully symbolic or numerical evaluation.

³The `norm` function is a helper which detects the number of terms N in a sum and divides by $\sqrt{N}$. See Appendix B for discussion of why this choice of oracle works.

⁴The `»` combinator conjugates operators, $A \gg B = A^\dagger B A$. In this case, conjugation by the Quantum Fourier Transform (QFT) operator `qft(X, Z)` simply exchanges X and Z in symbolic expressions.

⁵The function `measure` performs a projective-valued measurement (PVM), returning a post-measurement state and the measured outcome. The state `char(Z,0)` is $|0\rangle$, or rather, the functional $\pi_0(0) = \langle 0 | \cdot | 0 \rangle$.

⁶This `sample` is a random multiple of p/d . The post-processing is very simple; we just take `d` over the least common multiple of $O(\log(d))$ measurements, so period-finding requires seven lines:

$$p = d / \text{lcm}([\text{fourier_sample}(p, d) \text{ for } _ \text{ in range(trials)}]).$$

⁷We generalize the algorithm to arbitrary finite abelian groups in Appendix B.

⁸An elegant approach for $G = \mathbb{Z}_d$ is to use the generalized Clifford algebra $\mathcal{A}_d$ generated by Hermitian operators X, Z , satisfying $X^d = Z^d = I$ and the braiding relation $ZX = \omega_d XZ$, $\omega_d = e^{2\pi i/d}$.

⁹Usually, we introduce an ancilla as the output register of a function $f : G \rightarrow S$ whose level sets are precisely the cosets of pG . If we apply the oracle to a uniform superposition, we get a sum of orthogonal coset states. Here is the cute insight: we can define an operator in $\mathcal{A}_d$ which plays the same role! We just take a superposition of powers Z^{jq} for $q = d/p$.

¹⁰The QFT on the generalized Clifford algebra interchanges $X \leftrightarrow Z$. This turns a sum of powers Z^{jq} into a sum of powers X^{jq} .

¹¹Applying $\mathcal{F}[U_p]$ to $|0\rangle$ yields a superposition of states $|jq\rangle$, i.e. states in qG .

¹²Uniformly random multiples of q have a probability $P \approx 6/\pi^2 + O(1/d^2)$ of being coprime, so the probability of obtaining a jointly coprime set increases exponentially with the number of samples.

APPENDIX A: QISKIT REFERENCE IMPLEMENTATION

Here is a reference implementation of period-finding in Qiskit, using the standard circuit-based approach, and restricted to periods which are binary powers. Comments omitted:

```
import numpy as np
from fractions import Fraction
from math import lcm

from qiskit import QuantumCircuit
from qiskit.circuit.library import QFT, UnitaryGate
from qiskit_aer import AerSimulator

def oracle_unitary(f, n_x, n_f):
    dim = 2 ** (n_x + n_f)
    U = np.zeros((dim, dim), dtype=complex)

    def bits(x, n): return format(x, "b").zfill(n)
    def idx(x, y): return int(bits(x, n_x) + bits(y, n_f), 2)

    for x in range(2**n_x):
        for y in range(2**n_f):
            if y == 0:
                i, j = idx(x, 0), idx(x, f(x))
                U[i, j], U[j, i] = 1, 1
            else:
                i = idx(x, y)
                U[i, i] = 1

    return U

def period_find_qiskit(p, n_x=4, n_f=3, shots=2):
    N = 2**n_x
    assert p <= N and (p & (p - 1) == 0)

    def f(x): return x % p

    U = oracle_unitary(f, n_x, n_f)
    oracle_gate = UnitaryGate(U, label="Oracle")

    qc = QuantumCircuit(n_x + n_f, n_x)
    qc.h(range(n_x))
    qc.append(oracle_gate, range(n_x + n_f))
    qc.append(QFT(n_x, do_swaps=False), range(n_x))
    qc.measure(range(n_x), range(n_x))

    sim = AerSimulator()
    result, counts = sim.run(qc, shots=shots).result(), result.get_counts(qc)

    sample_ints = []
    for bitstr, c in counts.items(): sample_ints += [int(bitstr, 2)] * c

    denoms = [Fraction(s / (2**n_x)).denominator for s in sample_ints if s != 0]
    return lcm(*denoms) if denoms else 1, sample_ints, qc
```

APPENDIX B: A PHASE ORACLE FOR GENERAL ABELIAN GROUPS

Consider a finite abelian group G , which by the fundamental theorem of finite abelian groups can be written in the form

$$(1) \quad G \cong \mathbb{Z}_{d_1}^{m_1} \oplus \cdots \oplus \mathbb{Z}_{d_k}^{m_k}$$

with $\gcd(d_i, d_j) = 1$ for $i \neq j$. The associated algebra is a tensor product of Clifford algebras

$$(2) \quad \mathcal{A}_G = \mathcal{A}_{d_1}^{\otimes m_1} \otimes \cdots \otimes \mathcal{A}_{d_k}^{\otimes m_k}.$$

Now consider a “hidden” subgroup $H = \langle v_i \rangle \leq G$, where the v_i are “vectors” generating H . In standard presentations of the algorithm, we assume an oracle is given $f : G \rightarrow S$ such that

$$(3) \quad f(g) = f(g') \iff g - g' \in H.$$

We use a second register to record the output values. Another way to implement this idea is to record the level sets in the phase,

$$(4) \quad U_\chi |g\rangle = \chi(g) |g\rangle$$

for multiplicative functions $\chi(g + g') = \chi(g)\chi(g')$ such that $\chi(g) = \chi(g')$ just in case $g - g' \in H$.

The multiplicative functionals $\chi : G \rightarrow \mathbb{C}$ are called *characters* of G , and are enumerated by the dual group $\tilde{G}$; there is a dual subgroup $\tilde{H}$ of characters which are constant (but not necessarily distinct) on cosets of H . Thus, we can consider linear combinations

$$(5) \quad f = \sum_{\tilde{h} \in \tilde{H}} c_{\tilde{h}} \chi_{\tilde{h}} \implies U_f = \sum_{\tilde{h} \in \tilde{H}} c_{\tilde{h}} Z_{\tilde{h}}$$

where $Z_{\tilde{h}} |g\rangle = \chi_{\tilde{h}}(g) |g\rangle$ is a suitable product of “clock” operators. Performing a QFT on U_f transforms characters in the phase into computational basis states, or in operator language, exchanges $Z_{\tilde{h}}$ for

$$(6) \quad X_{\tilde{h}} |0\rangle = |\tilde{h}\rangle \implies \mathcal{F}[U_f] |0\rangle = \sum_{\tilde{h} \in \tilde{H}} c_{\tilde{h}} |\tilde{h}\rangle.$$

This is a uniform distribution over $\tilde{H}$ provided $|c_{\tilde{h}}|^2 = |\tilde{H}|^{-1}$. Thus, we have a single-register variant of the Fourier sampling subroutine for the abelian Hidden Subgroup Problem.

As an example, consider $G = \mathbb{Z}_d$ and a subgroup $H = pG$, the generator is p and the dual generator is $q = d/p$. The characters in $\tilde{H}$ and corresponding phase functions have the form

$$(7) \quad \chi_{jq}(x) = \omega_d^{jqx} \implies f = \sum_{j=0}^{p-1} c_j \chi_{jq}.$$

Given that Z also acts as a phase, with $Z|k\rangle = \omega_d^k |k\rangle$, we can write

$$(8) \quad U_f = \sum_{j=0}^{p-1} c_j Z^{jq}.$$

Taking the QFT exchanges $Z \leftrightarrow X$ in the Clifford algebra, leading to

$$(9) \quad \mathcal{F}[U_f] = \sum_{j=0}^{p-1} c_j X^{jq}.$$

Provided $|c_j|^2 = p^{-1}$, applying $\mathcal{F}[U_f]$ to $|0\rangle$ and measuring in the Z basis will yield an element of qG uniformly at random. Above, we chose $c_j = p^{-1/2}$ for simplicity.

REFERENCES

- [1] P. W. Shor, *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*, SIAM Journal on Computing **26**(5), 1484–1509 (1997).
- [2] A. M. Childs and W. van Dam, *Quantum algorithms for algebraic problems*, Reviews of Modern Physics **82**(1), 1–52 (2010). arXiv:0812.0380.
- [3] Qiskit Development Team, *Qiskit Textbook: Shor’s Algorithm*, <https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/shors-algorithm> (accessed December 2025).
- [4] D. Wakeham, *Introduction to yaw*, <https://torsor.io/assets/library/yaw/yaw011.html> (2025).
- [5] D. Wakeham, *The Structure and Interpretation of Quantum Programs I: Foundations*, arXiv:2509.04527 (2025).

TORSOR LABS, VANCOUVER, BC, CANADA